[5] Sharding consists in breaking down data, users or transactions (or all of them) in groups, each of which is assigned to a different machine/cluster/data-center. This balances the loads and allows to use smaller, cheaper servers and databases for each group, ultimately allowing applications to scale better and in a cheaper way.

[6] Relaxing the temporal requirement on consistency. We'll explain a bit more later in the chapter.

[7] For instance, https://www.manning.com/books/principles-of-cloud-design or https://www.manning.com/books/progressive-web-apps, which also has a nice chapter on caching.

[8] Using techniques like natural language processing, text analysis, computational linguistics to automatically identify the attitude of one or more subjects with respect to some topic.

[9] HTTP status code for "Service unavailable".

[10] The set of possible inputs for the function.

[11] As said, we are restricting only to fortune top 100 companies in this example.

[12] A cache's eviction policy is used to decide which element should be purged from a full cache to make room for a new entry.

[13] Ideally, all http responses should add `Cache-Control, Expires` and `Last-Modified` headers to make sure the resources are taken from cache as much as possible.

[14] Not all data can be shared without limits. That's why there are private caches, where data is shared with a single user/IP, and public cache (used for example for static, anonymous content) shared with anyone interested in the resource.

[15] We'll pick this example up again in section 7.7: figure 7.12 illustrates it.

[16] A software component shown in figures 7.1 and 7.2.

[17] Of course, there are many techniques, including machine learning, that can provide good predictions, but always with some degree of error – especially high in field like this were the users' behavior can change quickly and unexpectedly.

[18] Check chapter 2 for a refresher.

[19] Technically, as we have seen, it's more correct to say that the amortized time for `n` operations is `O(n)`, so individual calls usually takes constant time, but some of them might and will take longer, up to `O(n)`.

[20] There is also more to the story: if we do the opposite, then the way we link from a linked list node to the hash table entry will be tied to the implementation of the hash table: it could be an index for open addressing, or a pointer if we use chaining. This coupling to an implementation is neither good design nor, often, possible, as you usually can't access standard libraries internals (for good reason!).

[21] Remember we still need to include the time for computing each hash values for the entry we look up – see appendix C for more on this topic.

[22] This example is purely for illustration: in all likelihood, cache, DBs, web servers etc… for such a retailer will be sharded geographically, exactly to exploit this locality of preferences.